Темы занятия

• Определение свойств
• Свойства для чтения и записи
• Вычисляемые свойства
• Модификаторы доступа в свойствах
• Автоматические свойства
• Блок `init`
• Сокращённая запись свойств
• Модификатор `required`
• Свойства структур и классов
• Перегрузка методов

Домашние задания

Постановка

1. Выполните **индивидуально Задание** 1.
2. Выполните **индивидуально Задание** 2.
3. Выполните **совместно** в рабочей группе **Задание** 3.
4. Сформулируйте **список вопросов**, возникших в ходе выполнения **заданий** 1, 2 и 3.
5. **Передайте** список вопросов **Лидеру** группы.
6. (**Выполняет Лидер группы**) **Оформите Отчёт** о проделанной работе. Прикрепите список вопросов от каждого участника рабочей группы к Отчёту. (Требования к отчёту см. ниже.)
7. (**Выполняет Лидер группы**) **Отправьте Отчёт** о проделанной работе на указанную на занятии электронную почту.

Задание 1

Задание для разминки. Реализуйте класс `StudentProgress` по требованиям лабораторной работы (см. `//TODO: ⚠️`):

```
public class StudentProgress
{
    //TODO: ⚠️ 1. Свойство string — КОРОТКАЯ запись (автосвойство), обязательное при создании ⚠️
    //    Например: Subject — название предмета
```

```
    //TODO: ⚠️ 2. Свойство string — ПОЛНАЯ запись (с get и set, с проверкой на пустую строку) ⚠️
    //    Например: TeacherName — имя преподавателя

    //TODO: ⚠️ 3. Свойство int — ПОЛНАЯ запись, НЕ допускает отрицательных значений ⚠️
    //    Например: SemesterPoints — баллы за семестр

    //TODO: ⚠️ 4. Вычисляемое свойство bool — true, если int-свойство меньше 25 ⚠️
    //    Например: NeedsRetake => SemesterPoints < 25

    //TODO: ⚠️ 5. string GetInfo() — строка со значениями всех свойств ⚠️

    //TODO: ⚠️ 6. string GetInfo(bool includeIntProperty) — та же строка,
    //         но int-свойство включается только при includeIntProperty == true ⚠️
}
```

Проверьте:

1. `progress.SemesterPoints = -10;` должно приводить к исключению (или не менять значение — выберите один подход и обоснуйте).
2. `progress.GetInfo(false)` не должен содержать баллы.
3. `new StudentProgress()` без `Subject` не должен компилироваться.
4. Класс должен собираться и демонстрироваться в `Main`.
5. **Дополнительно (по желанию):** объявите ту же структуру данных как `readonly struct StudentProgressRecord` с теми же свойствами и убедитесь, что свойства структур объявляются так же, но `set` в `readonly struct` запрещён, а `protected` для членов структуры не компилируется.

Задание 2

Индивидуально. Ответьте **одной строкой кода** на каждый вопрос и проверьте ответ в проекте:

1. Как объявить автоматическое свойство `Group` типа `string` с начальным значением `"не распределён"`?
2. Как объявить свойство `Age`, которое можно читать снаружи, а менять — только внутри класса?
3. Как объявить вычисляемое свойство `IsJunior` (возраст меньше 20)?
4. Как объявить свойство `CardNumber`, которое задаётся только при создании объекта?

5. Как объявить обязательное свойство `FullName`, без которого объект не создать?
6. Почему нельзя объявить два метода `GetInfo()` с разными типами возврата?
7. Почему у свойства в `readonly struct` не может быть аксессора `set`?

Ответы к заданию 2 (для самопроверки)

1. `public string Group { get; set; } = "не распределён";`
2. `public int Age { get; private set; }`
3. `public bool IsJunior => Age < 20;`
4. `public string CardNumber { get; init; }`
5. `public required string FullName { get; init; }`
6. Тип возврата не входит в сигнатуру метода, поэтому обе версии имеют одну и ту же сигнатуру — ошибка CS0111.
7. `readonly struct` запрещает изменять свои поля после создания, поэтому `set` не компилируется: допустимы `init` (значение задаётся при создании) или свойство только для чтения.

Задание 3 (групповое)

Проект

Мы **продолжаем проект маркетплейса** (занятия 2 и 3). Каталог, доступ к данным и печать уже готовы; на этом занятии заменяем методы `Get...` / `Set...` на **свойства** и добавляем **перегрузки** `GetInfo`.

Правила проекта:

1. Один тип — один файл; пространства имён `Marketplace.Domain` и `Marketplace.Services`.
2. Имена свойств — существительные с большой буквы (`Price`, `IsBudget`), имена методов — глаголы (`Calculate`, `GetReport`).
3. Названия типов из занятий 2 и 3 не меняются: `Product`, `CustomerProfile`, `Cart`, `CartItem` (у `CartItem` меняется только вид: класс становится `readonly struct`).

Распределение задач по участникам

Участник 1 — класс `Product` (`Product.cs`) — главный по теме занятия

Класс должен содержать **все четыре вида свойств** из лабораторной и **две перегрузки** `GetInfo`:

```
namespace Marketplace.Domain;

public class Product
{
    // 1. string — сокращённая (автоматическая) запись, обязательна при
создании
    public required string Title { get; init; }

    // 2. string — полная запись с проверкой на пустоту
    private string article = string.Empty;
    public string Article
    {
        get => article;
        set
        {
            if (string.IsNullOrWhiteSpace(value))
                throw new ArgumentException("Артикул обязателен",
nameof(value));
            article = value;
        }
    }

    // 3. int — полная запись, не допускает отрицательных значений
    private int price;
    public int Price
    {
        //TODO: ⚠️ get и set с проверкой value < 0 ⚠️
    }

    // 4. bool — вычисляемое свойство: true, если Price < 25
    //TODO: ⚠️ public bool IsBudget => ... ⚠️

    // 5. Перегрузки
    //TODO: ⚠️ public string GetInfo() => GetInfo(true); ⚠️
    //TODO: ⚠️ public string GetInfo(bool includeIntProperty) { ... } ⚠️

    // 6. Статус меняется только внутри класса
    public string Status { get; private set; } = "В наличии";

    //TODO: ⚠️ public void ChangeStatus(string newStatus) ⚠️
    //TODO: ⚠️ private static void EnsureStatus(string value) ⚠️
}
```

Требуется дополнительно:

1. Свойство `Status` с `private set` (статус меняется только внутри класса).
2. Метод `ChangeStatus(string newStatus)` с **приватной** проверкой допустимых значений: `"В наличии"`, `"Нет в наличии"`, `"Под заказ"` (список допустимых значений храните в приватном статическом поле).
3. Убедиться, что `product.Price = -10;` и `product.Status = "Продан";` не компилируются, а `product.ChangeStatus("Продано");` выбрасывает исключение.

Участник 2 — класс `CustomerProfile` (`CustomerProfile.cs`)

Требуется:

1. `FullName` — автоматическое свойство, `required` + `init`.
2. `Phone` — автоматическое свойство с инициализатором `string.Empty`.
3. `Email` — **полная** запись: сеттер проверяет, что строка содержит `@` (приватный метод `EnsureValidEmail`).
4. `Rating` — `int`, полная запись с проверкой диапазона `0..5`, резервное поле `rating`.
5. Вычисляемое свойство `IsTrusted => Rating >= 4`.
6. Перегруженные методы:
   - `string GetInfo()`;
   - `string GetInfo(bool includeRating)`;
   - `string GetContactInfo(bool includePhone)`.
7. **1-2 приватных** метода (`EnsureValidEmail`, `NormalizePhone`).

```
namespace Marketplace.Domain;

public class CustomerProfile
{
    public required string FullName { get; init; }
    public string Phone { get; set; } = string.Empty;

    //TODO: ⚠️ Email — полная запись с EnsureValidEmail ⚠️
    //TODO: ⚠️ Rating — полная запись с проверкой 0..5 ⚠️
    //TODO: ⚠️ public bool IsTrusted => Rating >= 4; ⚠️
    //TODO: ⚠️ public string GetInfo() и GetInfo(bool includeRating) ⚠️
    //TODO: ⚠️ public string GetContactInfo(bool includePhone) ⚠️
    //TODO: ⚠️ private static void EnsureValidEmail(string value) ⚠️
    //TODO: ⚠️ private static string NormalizePhone(string value) ⚠️
}
```

Участник 3 — скидки и расчёт (`Discount.cs`)

Требуется:

1. Класс `Discount` со свойствами:
   - `Name` — сокращённая запись, `required`;
   - `Percent` — `decimal`, полная запись с проверкой `0..100`;
   - `MinSum` — `int`, полная запись с проверкой `>= 0` (минимальная сумма заказа);
   - `ValidUntil` — `DateTime` с `init`;
   - `IsActive` — вычисляемое `bool` (`ValidUntil > DateTime.Now`).
2. Метод `int Calculate(int total)` — цена после скидки с учётом минимальной суммы; внутри **1-2 приватных** метода (`ApplyDiscount`, `EnsurePercent`).
3. **Перегрузка** `int Calculate(int total, decimal percent)`.
4. Показать, что передача недопустимых значений через свойства не проходит.

```
namespace Marketplace.Domain;

public class Discount
{
    public required string Name { get; init; }
    public DateTime ValidUntil { get; init; }

    //TODO: ⚠️ Percent — decimal, полная запись, 0..100 ⚠️
    //TODO: ⚠️ MinSum — int, полная запись, >= 0 ⚠️
    //TODO: ⚠️ public bool IsActive => ValidUntil > DateTime.Now; ⚠️

    //TODO: ⚠️ public int Calculate(int total) ⚠️
    //TODO: ⚠️ public int Calculate(int total, decimal percent) ⚠️

    //TODO: ⚠️ private static int ApplyDiscount(int total, decimal percent)
⚠️
    //TODO: ⚠️ private static void EnsurePercent(decimal value) ⚠️
}
```

Участник 4 — позиция корзины и сервис оформления заказа (`CartItem.cs`, `CheckoutService.cs`)

Требуется:

1. `readonly struct CartItem` — позиция корзины со свойствами:
   - `ProductId`, `Quantity`, `PricePerUnit` — только для чтения, задаются конструктором;
   - `Total` — вычисляемое свойство (`PricePerUnit * Quantity`);
   - конструктор проверяет, что количество положительное, а цена не отрицательная.

> ℹ️ На занятии 2 позиция корзины была классом. Теперь оформляем её как `readonly struct` : данные позиции после создания не меняются, а стоимость считается свойством.

2. Сервисный класс `CheckoutService` без состояния (только методы):
   - `int GetTotal(IEnumerable<CartItem> items)` — сумма `Total` по всем позициям;
   - `IReadOnlyList<CartItem> FindCheaperThan(IEnumerable<CartItem> items, int maxPrice)` - найти позиции дешевле заданной цены;
   - перегруженные `string GetReport(IEnumerable<CartItem> items)` (короткая сводка) и `string GetReport(IEnumerable<CartItem> items, bool includeLines)` (сводка со строками позиций).
3. **1-2 приватных** метода: `IsAvailable(CartItem item)` и `PrepareLine(CartItem item)` - "доступна ли позиция для покупки?" и "Подготовить красивую строку для отчёта или чека на основе товарной позиции в корзине".
4. Сервис не должен обращаться к закрытым полям чужих классов — только к открытым свойствам и методам.

```
namespace Marketplace.Domain;

public readonly struct CartItem
{
    public int ProductId { get; }
    public int Quantity { get; }
    public int PricePerUnit { get; }

    //TODO: ⚠️ Конструктор с проверками ⚠️

    //TODO: ⚠️ public int Total => PricePerUnit * Quantity; ⚠️
}
```

```
using Marketplace.Domain;

namespace Marketplace.Services;

public class CheckoutService
{
    public int GetTotal(IEnumerable<CartItem> items)
    {
        //TODO: ⚠️ Сумма Total по всем позициям ⚠️
        return 0;
    }
```

```
    //TODO: ⚠️ public IReadOnlyList<CartItem> FindCheaperThan(...) ⚠️
    //TODO: ⚠️ public string GetReport(IEnumerable<CartItem> items) ⚠️
    //TODO: ⚠️ public string GetReport(IEnumerable<CartItem> items, bool
includeLines) ⚠️
    //TODO: ⚠️ private bool IsAvailable(CartItem item) ⚠️
    //TODO: ⚠️ private string PrepareLine(CartItem item) ⚠️
}
```

Компоновка (задача Лидера)

Лидер рабочей группы:

1. собирает все классы в один проект: по одному типу на файл, пространства имён `Marketplace.Domain` и `Marketplace.Services`;
2. пишет точку входа (с явно прописанным методом **Main** и служебными методами для каждого этапа демонстрации), где:
   - создаёт **3 товара** и **2 покупателя** (обязательные свойства заданы в инициализаторах);
   - собирает `List<CartItem>` и печатает **отчёты** через `CheckoutService`;
   - показывает работу **вычисляемого свойства** `IsBudget` и обоих вариантов `GetInfo`;
   - применяет **скидку** через `Discount.Calculate` и показывает **обе перегрузки**;
   - в `try/catch` демонстрирует попытку записать **отрицательную цену**, **пустой артикул** и **недопустимый рейтинг** (в catch блоке делаем вывод в консоль сообщения с описанием ситуации);
   - демонстрирует, что после создания объекта **нельзя изменить** `Title` (`init`) и `Status` (`private set`) - текст предупреждения от редактора кода записываем в отчёт - в листинге оставляем комментарий с кодом попытки вызвать сеттер и дополнительным комментарием "**Иллюстрация попытки вызвать сеттер для неизменяемого свойства**";
3. проверяет единообразие названий и стиля;
4. оформляет **отчёт**.

Рекомендации по выполнению

1. **Один класс — один файл**, имена свойств — существительные с большой буквы (`Price`, `IsTrusted`), имена методов — глаголы (`Calculate`, `GetReport`).
2. **Проверка в сеттере или в конструкторе?** Выберите один подход на группу и придерживайтесь его: либо сеттер бросает исключение, либо отклоняет значение. В отчёте объясните выбор.

3. **`required` + `init`** — хорошее сочетание для «паспортных» данных (`Title`, `FullName`), которые не должны меняться после создания.
4. **Вычисляемое свойство** не должно иметь сеттера и не должно быть «дорогим»: только простая формула по другим свойствам.
5. **Перегрузки** не должны дублировать код: короткая версия всегда вызывает полную.
6. **Каждый участник** пишет либо небольшой класс, либо как минимум один серьёзный публичный метод сервиса с 1-2 приватными методами внутри.

Требования к отчёту

Отчёт оформляет **Лидер группы**, файл — в формате Markdown или DOCX.

1. **Титульная часть:** номер занятия, тема, состав рабочей группы (3-4 человека, лидер отмечен).
2. **Текст исходного задания** — полностью, как он приведён в этом файле.
3. **Для каждого задания:** краткое описание выполненного решения (2-5 предложений: какое свойство как объявлено и зачем).
4. **Под каждым описанием — листинг решения** (код с комментариями) или указанное **имя файла с кодом**.
5. **Таблица свойств проекта:** имя свойства, тип, вид записи (авто / полная / вычисляемая / `init` / `required`), кто реализовал.
6. **Доказательства:** вывод программы для обоих вариантов `GetInfo`, сообщение об ошибке при попытке записать отрицательную цену, попытке изменить `init`-свойство.
7. **Список вопросов** от каждого участника группы. (опционально)
8. **Выводы:** в каких случаях свойство удобнее открытого поля, а в каких — метода; где перегрузка упростила вызов.

Критерии оценки группы:

- проект собирается и запускается без правок;
- присутствуют все виды свойств из задания и обе перегрузки `GetInfo`;
- недопустимые значения не попадают в объект;
- каждый участник написал свою часть;
- оформление отчёта соответствует требованиям выше.

==**Куда присылать работы:** ==
moysklad-students-homework@mail.ru

```
moysklad-students-homework@mail.ru
```

Тема письма:

```
$"ООП 2026 Домашняя работа 4 {номер_учебной_группы} {ФИО}"
```

Что в письме:

- `.cs` файлы классов проекта ИЛИ архив с проектом (без мёртвого кода - только то, что используется в демонстрации).
- `readme.md` - отчёт.

Никаких скриншотов - только текст.

Срок сдачи — до вечера пятницы.

Материалы

Свойства

Свойство — это член класса, который **выглядит как поле**, но при чтении и записи **выполняет код**.

```
student.Age = 20;        // похоже на запись в поле...
int a = student.Age;     // ...и на чтение поля
```

Но за этой записью может стоять проверка, вычисление или обращение к другому хранилищу.

Зачем нужны свойства:

- сохраняют привычный синтаксис полей;
- позволяют **проверять** данные при записи (в отличие от открытого поля);
- скрывают **детали хранения** (например, возраст лучше хранить датой рождения).

Определение свойства

Свойство состоит из **аксессоров**:

- **`get`** — вызывается при чтении;

- **`set`** — вызывается при записи (внутри `set` доступно ключевое слово `value`).

```
public class Student
{
    public string Name { get; set; }   // свойство для чтения и записи
}
```

Вопрос для закрепления: какой аксессор вызывается, когда мы пишем `student.Name = "Анна";`?

> **Ответ:** `set`; при чтении `student.Name` вызывается `get`.

Свойства для чтения и записи

Полная запись с проверкой значения и собственным полем-хранилищем:

```
public class Student
{
    private int age;

    public int Age
    {
        get
        {
            return age;
        }
        set
        {
            if (value < 14 || value > 100)
                throw new ArgumentOutOfRangeException(nameof(value),
"Недопустимый возраст");

            age = value;
        }
    }
}
```

Здесь `age` — **резервное поле** (backing field): именно оно хранит значение, а свойство выполняет роль «двери» к нему.

Вопрос для закрепления: можно ли в сеттере записать в `value` другое значение?

> **Ответ:** нет. `value` — неявный параметр сеттера, доступный только для чтения; `value = ...` внутри `set` не компилируется.

Вычисляемые свойства

Вычисляемое свойство не хранит отдельного значения — оно **вычисляется** при каждом чтении. У него нет сеттера.

```
public class Student
{
    public string Name { get; set; } = string.Empty;
    public string Group { get; set; } = string.Empty;

    private int age;

    public int Age
    {
        get => age;
        set => age = value >= 0 ? value : throw new
ArgumentOutOfRangeException(nameof(value));
    }

    // Вычисляемое свойство: true для студентов младше 20 лет
    public bool IsJunior => Age < 20;

    // Вычисляемое свойство для печати
    public string Info => $"{Name}, {Age} лет, группа {Group}";
}
```

Читать можно: `student.IsJunior`, а писать — нельзя: `student.IsJunior = true;` не скомпилируется.

Вопрос для закрепления: почему у вычисляемого свойства нет аксессора `set`?

> **Ответ:** значение не хранится отдельно — оно каждый раз вычисляется из других свойств, поэтому записывать в него нечего.

Модификаторы доступа в свойствах

Аксессоры могут иметь **разные** модификаторы: например, читать разрешено всем, а менять — только внутри класса.

```
public class Student
{
    public int Age { get; private set; }   // set доступен только внутри
Student
```

```
    public string Name { get; set; }       // открыто и чтение, и запись
}
```

Правила:

- модификатор можно указать **только у одного** аксессора из пары `get` / `set`;
- этот модификатор должен быть **строже** модификатора самого свойства (например, `private` при `public` свойстве).

```
student.Age = 25;   // ошибка: set закрыт
int age = student.Age; // OK
```

Вопрос для закрепления: можно ли объявить свойство `private int Age { get; public set; }`?

> **Ответ:** нет: модификатор аксессора должен быть **строже** модификатора свойства. Здесь `public` шире, чем `private`, — компилятор выдаст ошибку.

Автоматические свойства

Если проверки не нужны, поле-хранилище можно не писать — компилятор создаст его сам.

```
public class Student
{
    public string Name { get; set; } = "Без имени";   // автоматическое
свойство с начальным значением
    public string Group { get; set; } = string.Empty;
    public int Age { get; set; }
}
```

Свойство **только для чтения** с автоматическим хранилищем:

```
public class Statement
{
    public Guid Id { get; } = Guid.NewGuid();   // значение задаётся один
раз
}
```

Вопрос для закрепления: сколько полей компилятор создаст для класса с тремя автоматическими свойствами?

> **Ответ:** три — по одному скрытому резервному полю на каждое автосвойство.

Блок `init`

`init` — «сеттер для создания объекта». Значение можно задать **только при инициализации**, а дальше свойство становится доступным только для чтения.

```
public class StudentCard
{
    public string Number { get; init; } = string.Empty;
    public int Course { get; init; }
}

var card = new StudentCard { Number = "Б-2417", Course = 2 };  // OK
// card.Course = 3;                                            // ошибка компиляции
```

`init` удобен для «неизменяемых после создания» данных: номер билета, дата зачисления, идентификатор.

Вопрос для закрепления: чем `init` отличается от `private set`?

> **Ответ:** `init` вызывается только при создании объекта (в инициализаторе или конструкторе), а `private set` — из любого места внутри класса, в том числе позже.

Сокращённая запись свойств

Свойство, которое только читает выражение, можно записать в одну строку:

```
public class Student
{
    public string Name { get; set; } = string.Empty;
    public string Group { get; set; } = string.Empty;

    // Сокращённая (expression-bodied) запись: вместо { get { return ...; } }
    public string FullInfo => $"{Name} (группа {Group})";

    // Сокращённая запись аксессоров у свойства с хранилищем
    private int age;
    public int Age
    {
        get => age;
        set => age = value;
    }
}
```

Вопрос для закрепления: в каких случаях допустима запись `public bool IsJunior => Age < 20;`?

> **Ответ:** когда свойство только для чтения и его значение выводится из других членов; тип выражения (`bool`) совпадает с типом свойства.

Модификатор `required`

С C# 11 свойство можно пометить как **обязательное при создании объекта**: если его не задать в инициализаторе или конструкторе, будет ошибка компиляции.

```
public class Student
{
    public required string FullName { get; set; }   // обязательно при
создании
    public string Group { get; set; } = "не распределён";
    public int Age { get; set; }
}

var student = new Student { FullName = "Анна Петрова", Age = 19 };  // OK
// var wrong = new Student { Age = 19 };                            //
ошибка: нет FullName
```

`required` обычно сочетают с `init`, чтобы значение нельзя было поменять позже:

```
public required string FullName { get; init; }
```

Вопрос для закрепления: какой ошибкой закончится `new Student()` без инициализатора, если `FullName` помечено `required`?

> **Ответ:** ошибкой компиляции CS9035: обязательное свойство `FullName` должно быть задано в инициализаторе объекта или в конструкторе с `[SetsRequiredMembers]`.

Свойства структур и классов

Свойства объявляются **одинаково** и у класса, и у структуры: это всё те же аксессоры `get` / `set`, автосвойства, `init` и `required`.

```
// Студенческий билет: значимый тип, который удобно сделать неизменяемым
public readonly struct StudentCard
{
    public string Number { get; init; }        // задаётся при создании
    public int Course { get; init; }
```

```
    public StudentCard(string number, int course)
    {
        Number = number;
        Course = course;
    }

    // Вычисляемое свойство
    public bool IsSenior => Course >= 4;
}
```

Отличия структуры от класса, о которых нужно помнить:

	Класс	**Структура**
Где живут значения свойств	объект в куче	сама переменная (стек или внутри другого значения)
Автосвойство с `get; set;`	обычное поведение	работает, но `readonly struct` запрещает `set`
Модификаторы аксессоров	`private`, `protected`, `internal`	`protected` и `protected internal` **запрещены**
`required`	можно	можно, но объект придётся создавать через конструктор или инициализатор
Изменение после создания	обычное дело	для `readonly struct` значения не меняются

```
var card = new StudentCard("Б-2417", 2);
// card.Course = 3;   // ошибка: readonly struct и init
```

Вопрос для закрепления: можно ли объявить свойство с модификатором `protected` в структуре?

> **Ответ:** нет: структура не наследуется, поэтому `protected`, `protected internal` и `private protected` для её членов не компилируются.

Перегрузка методов

Перегрузка — несколько методов с **одним именем** и **разными списками параметров** внутри одного класса.

```
public class Student
{
    public string GetInfo()
    {
        return GetInfo(true);   // делегируем перегруженной версии
    }

    public string GetInfo(bool includeAge)
    {
        return includeAge
            ? $"{FullName}, возраст: {Age}"
            : $"{FullName}";
    }

    public required string FullName { get; init; }
    public int Age { get; set; }
}
```

Вызов:

```
var student = new Student { FullName = "Анна Петрова", Age = 19 };
Console.WriteLine(student.GetInfo());         // Анна Петрова, возраст: 19
Console.WriteLine(student.GetInfo(false));    // Анна Петрова
```

Что делает метод «другим»

Сигнатура метода — это **имя + типы и порядок параметров**. Отличаться должны именно они:

Разрешено	Запрещено
`Print(string text)` и `Print(int number)`	`Print(string text)` и `Print(string label)` — **имена не важны**
`Print(string text)` и `Print(string text, bool upper)`	`int Print()` и `string Print()` — **тип возврата не часть сигнатуры**
`Print(string text)` и `Print(params string[] texts)`	

```
public class Journal
{
    // Перегрузки по типу параметра
    public void AddMark(int mark) => AddMark("без предмета", mark);
```

```
    // Перегрузки по количеству параметров
    public void AddMark(string subject, int mark)
    {
        if (mark < 0 || mark > 5)
            throw new ArgumentOutOfRangeException(nameof(mark));

        // ... сохранение оценки
    }
}
```

Вопрос для закрепления: можно ли объявить два метода `GetInfo()` с разными типами возврата и почему?

> **Ответ:** нет: тип возврата не входит в сигнатуру, поэтому такая пара — ошибка CS0111 «метод с такой сигнатурой уже объявлен».

Как выбирается нужная перегрузка

Компилятор выбирает **наиболее подходящую** версию по типам аргументов. Если подходящих несколько и выбрать нельзя — ошибка «неоднозначный вызов».

```
public void Show(int value) { }
public void Show(double value) { }

Show(5);      // int — точное совпадение
Show(5.0);    // double — точное совпадение
Show(5L);     // long: в int неявного преобразования нет, остаётся double
```

Вопрос для закрепления: зачем нужна перегрузка, если можно было объявить методы `GetInfo` и `GetInfoWithAge`?

> **Ответ:** одно имя проще запомнить и удобнее вызывать: короткая версия вызывает полную, а вызывающий код не думает о том, как называется «та же» операция с параметром.

Вопросы для самопроверки (ответ — одна строка кода)

1. Как объявить вычисляемое свойство `IsJunior`, истинное для студентов младше 20 лет?

```
public bool IsJunior => Age < 20;
```

2. Как разрешить чтение возраста всем, а изменение — только внутри класса?

```
public int Age { get; private set; }
```

3. Как сделать так, чтобы группу можно было задать только при создании объекта и никогда не менять позже?

```
public string Group { get; init; }
```

4. Как объявить перегрузку `GetInfo()`, которая просто вызывает версию с параметром `true`?

```
public string GetInfo() => GetInfo(true);
```

Что запомнить:

- свойство — это «поле с кодом»: `get` при чтении, `set` при записи;
- проверка данных живёт в сеттере, тогда «плохое» значение просто не попадёт в объект;
- вычисляемое свойство не хранит значение и не имеет `set`;
- `private set` — читают все, меняет только сам класс; `init` — только при создании; `required` — объект без значения не создать;
- свойства у структуры объявляются так же, но в `readonly struct` нет `set`, а `protected` запрещён;
- перегрузка — это одно имя и **разные** списки параметров; тип возврата сигнатуру не меняет.